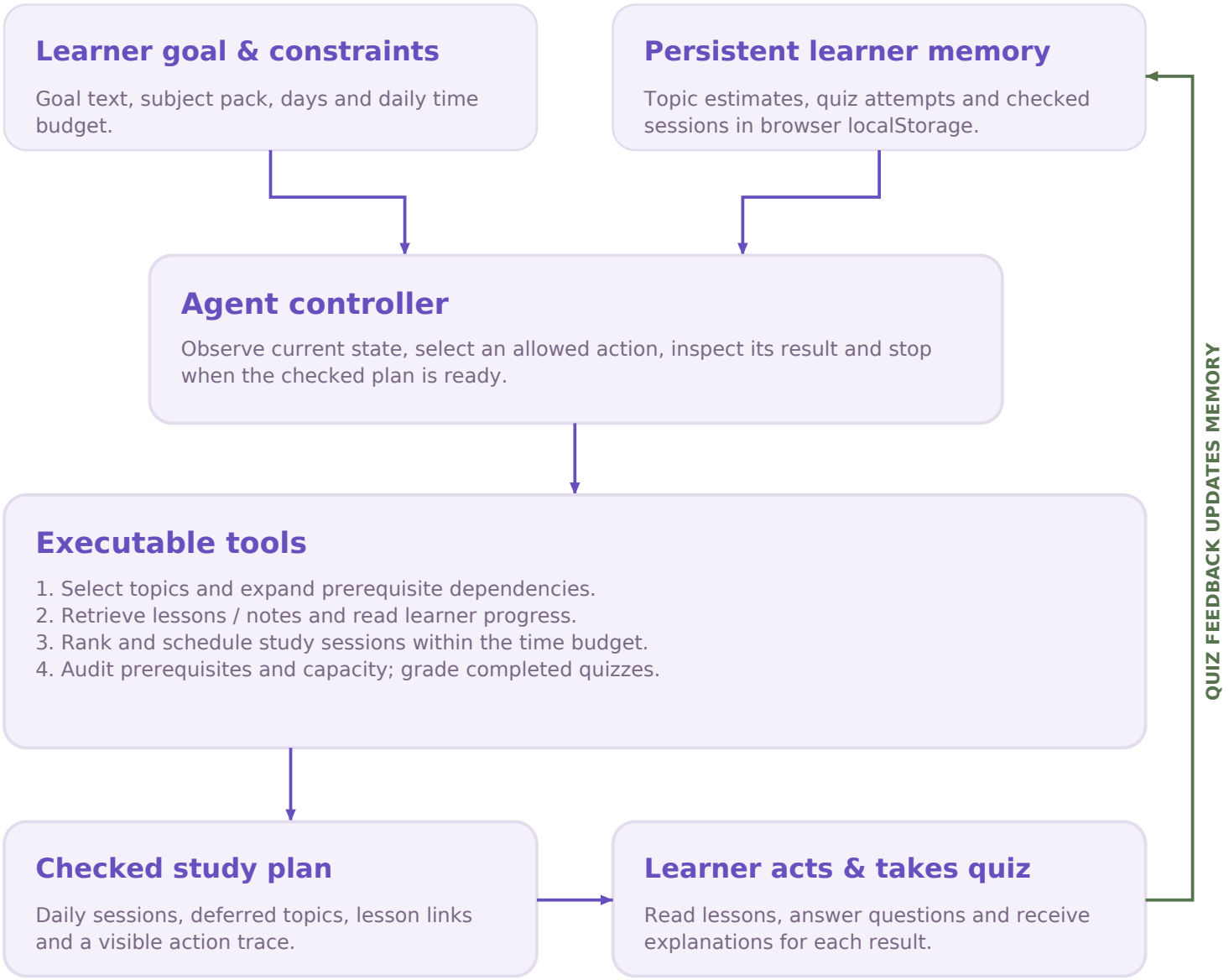
OPTION (I) / AGENTIC AGENT

StudyPilot

A goal-driven adaptive study agent. The local controller uses symbolic rules and a priority heuristic; an optional language-model coach uses bounded function calling.



Knowledge source: 12 built-in lessons, 36 original practice questions and a prerequisite graph. Optional plain-text notes are searchable but do not generate new quiz questions.

The feedback loop: a quiz changes the knowledge estimate; the agent then recalculates priorities and the study plan. Merely checking off a session does not increase the estimate.

DESIGN DETAILS

Policy, tools & data

A compact implementation that can be explained, inspected and reproduced.

01 DECISION POLICY

Priority = 3 x (1 - estimate) + 2 x goal_match + overdue_bonus

goal_match is 1 for an exact goal keyword match, otherwise 0. overdue_bonus is 0.5 when the last quiz is at least three days old. Prerequisites are ordered before dependent topics. A full lesson is deferred when it cannot fit; a scheduled lesson can be split across days.

02 FEEDBACK & STOPPING RULES

An untested topic has a 0.35 planning prior, displayed as untested. The first quiz estimate equals its score. Later estimates use 0.7 x new score + 0.3 x previous estimate. At 0.80 or above, a topic receives a shorter review session. These are demonstration heuristics, not calibrated measures of ability.

The local controller allows at most 8 steps and stops when the plan passes its checks. After a quiz it observes new evidence and starts a new planning run. The optional LLM coach is limited to 5 rounds, 8 tool calls and a 90-second overall timeout.

03 DATA SCHEMA

Entity	Main fields	Purpose
Goal	text, pack, days, minutes	Task and time constraints
Topic	id, title, prerequisites, minutes, summary, example, questions	Knowledge and dependency graph
Topic memory	mastery, lastReviewed, attempts	Recency-weighted quiz estimate
Quiz attempt	topicId, correct, total, score, at	Observed learner feedback
Plan / day / task	scope, deferred, days, used; topicId, mode, minutes, part	Scheduled learning actions
Trace step	step, action, observation	Inspectable action outcomes

04 OPTIONAL LANGUAGE-MODEL CONTROLLER

Browser request -> local Node server -> OpenAI Responses API -> allowlisted tool execution -> tool results returned to the model -> grounded explanation. Tools: lookup_lessons, inspect_progress and create_study_plan.

The API key stays in the server environment. No keys are included in source or browser storage. Requests to the AI coach send the question, goal and tool-selected study data to OpenAI. Local mode makes no model API calls.

SUBMISSION & DEMONSTRATION

Ready to explain

Repository, reproducible setup and a short viva walkthrough.

PROJECT REPOSITORY

https://github.com/kritee-saxena/AI_AGENT

RUN THE PROJECT

1. Install Node.js 22 or later, then download or clone the repository.
2. In the project directory run: npm start
3. Open: http://localhost:3000
4. Run automated checks with: npm test

TWO-MINUTE DEMONSTRATION

1. **Set a goal.** Enter "Learn CNN and pooling", choose AI fundamentals, 3 days and 45 minutes per day.
2. **Show autonomous planning.** Point out the machine-learning and neural-network prerequisites. Explain that daily limits and topic order are checked.
3. **Give feedback.** Take the CNN quiz. Use one incorrect answer to show the explanation and saved knowledge estimate.
4. **Show adaptation.** Open My progress, then Study plan. Point to the new feedback trace. Compare priorities before and after the quiz.
5. **Explain the implementation.** The browser runs a classical symbolic agent. The optional server adds actual LLM function calling when configured with an API key and available model.

VALIDATION & LIMITS

11 automated tests passed: prerequisites, capacity bounds, deferred work, quiz-driven replanning, score updates, retrieval, bad inputs, curriculum consistency, mocked LLM tool flow, call limits and local HTTP behavior.

A live model call was not run because no API key was configured. The LLM tests use a deterministic mock. Browser visual and interaction QA was not available in the authoring environment. The project is a teaching prototype, with a small fixed curriculum and device-local memory.

Submission category is proposed as option (i). If the marking rubric specifically requires an LLM, configure and demonstrate the optional coach. The faculty determines the final category and marks.

API reference: developers.openai.com/api/docs/guides/function-calling